

MACHINE-PAYMENTS REVENUE SPRINT

INNOVATE • BUILD • IMPACT

 DURATION

 FOCUS

 OBJECTIVE

W1T6

Object type Internship

Week 1

Task 1

Creation date Bugün

Created by  Sorest

Keywords Seçenekleri seç

Tasks

 [x402 Facilitator — Comprehensive Guide](#)

- [1. What is a Facilitator?](#)
- [2. Why Use a Facilitator?](#)
- [3. Facilitator Responsibilities](#)
- [4. End-to-End Interaction Flow](#)
 - [Step-by-Step Breakdown](#)
- [5. Choosing a Facilitator Path](#)
- [6. Live Facilitators](#)
- [7. Solana Duplicate Settlement Protection](#)
- [8. Restaurant Analogy](#)

 [Glossary](#)

 [x402'de Mandate, Intent vs Cart ve Authorization ≠ Settlement](#)

- [1. What is a Mandate?](#)
- [2. ★ Intent vs. Cart — Two Different Paradigms](#)
 - [🛒 Cart Model \(Traditional E-Commerce\) 💡 Intent Model \(x402\)](#)
 - [Comparison Table](#)
- [3. ★★ Authorization ≠ Settlement — Why Separate Layers?](#)
 - [Real-World Analogy: Hotel Check-in](#)
 - [How This Operates in x402](#)
- [4. Why Decouple Authorization and Settlement?](#)
 - [Detailed Breakdown](#)
 - [⚡ 1. Latency Optimization](#)
 - [💰 2. Metered / Pay-As-You-Go Pricing \(upto Scheme\)](#)
 - [🛡️ 3. Fraud Prevention & Risk Control](#)
 - [🏠 4. Batching Micropayments \(batch-settlement Scheme\)](#)
 - [5. Comparison of the 3 Payment Schemes](#)
 - [6. End-to-End Architecture Flow](#)
 - [7. Common Misconceptions](#)

 [Glossary](#)

 [A2A + x402: How Agents Communicate and Pay Each Other](#)

- [1. The Big Picture — In 3 Seconds](#)
- [2. What is A2A? — Simple Analogy](#)

Corporate Office Analogy

3.

★

Agent Card — The Agent's Business Card

Well-Known Endpoint Path

Agent Card Architecture

Example agent.json Document

Analogy: LinkedIn Profile

4.

★

Task Lifecycle — State Machine

State Diagram

Task State Descriptions

Message & Artifact Flow: Translation Task Example

Anatomy of an A2A Task Object

5.

★

★

How A2A and x402 Compose

Layered Architecture

Core Principle: A2A is Payment-Agnostic

6.

★

★

★

End-to-End Scenario: Agent Hires Another Agent

Step-by-Step Breakdown

7.

Enriching Agent Cards with x402 Metadata

8.

Comprehensive Mapping: Upwork vs. A2A + x402

10.

Architectural Summary

📖

Glossary

Tasks

POC

8h

x402 / AP2 / A2A landscape scan

DoD: 2-page landscape note + 3 candidate IoT use-cases, one paragraph each.

Open playbook

READ (IN THIS ORDER)

◦ x402 spec + how the facilitator model works (402 response → signed payment header → verify/settle → resource).

◦ AP2: what a mandate is (intent vs cart), why authorization is a separate layer from settlement.

◦ a2a-x402 extension: how the two compose. A2A basics: Agent Card, task lifecycle.

DO

◦ Clone github.com/coinbase/x402 , run a seller + buyer example end-to-end on Base Sepolia with your faucet USDC.

◦ Write the landscape note: how the pieces fit, what's mature vs rough, where Cloudflare fits (Workers, DO, Agents SDK).

◦ Sketch 3 candidate IoT use-cases. For each: buyer agent, seller agent, what's simulated vs real, price unit (\$/reading, \$/inference, \$/minute).



x402 Facilitator — Comprehensive Guide

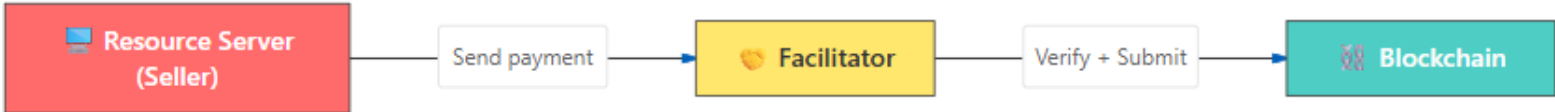


Source: docs.x402.org/core-concepts/facilitator

1. What is a Facilitator?

Facilitator = A service that verifies payments and submits them to the blockchain on behalf of the server.

The facilitator is an **optional but recommended** intermediary in the x402 protocol. It removes the need for resource servers (sellers) to maintain direct blockchain connectivity.



TIP

The facilitator **does not hold funds** and is **not a custodian**. It only performs verification and execution of onchain transactions based on signed payloads provided by clients.

2. Why Use a Facilitator?

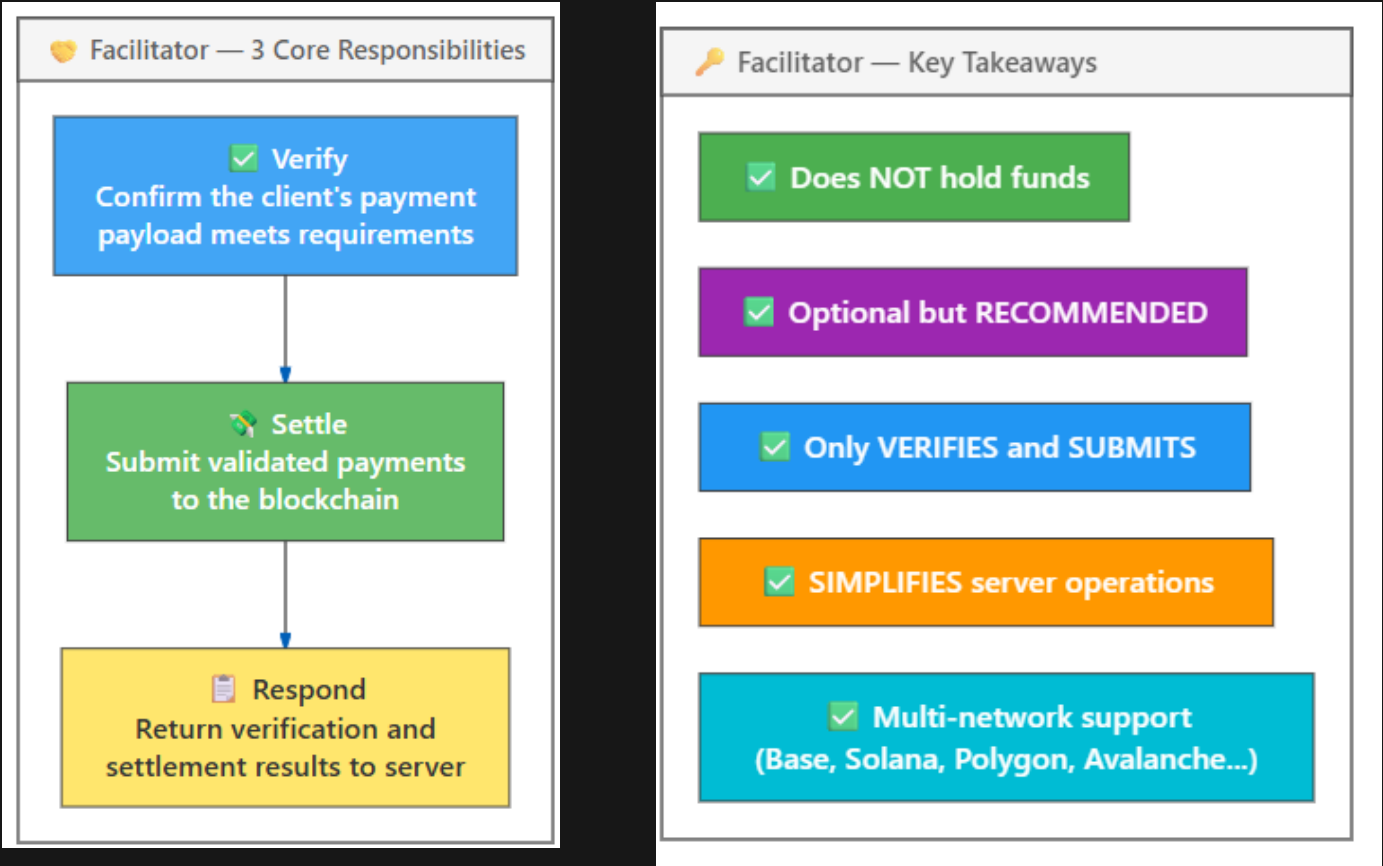
Without a facilitator, the resource server must:

- Run blockchain nodes
- Implement payment verification logic
- Manage gas fees and transaction monitoring

With a facilitator, the server simply says *"verify this payment and settle it"* — all blockchain complexity is abstracted away.

Benefit	Description
Reduced complexity	Servers don't interact directly with blockchain nodes
Protocol consistency	Standardized verify/settle flows across services
Faster integration	Start accepting payments with minimal blockchain-specific code

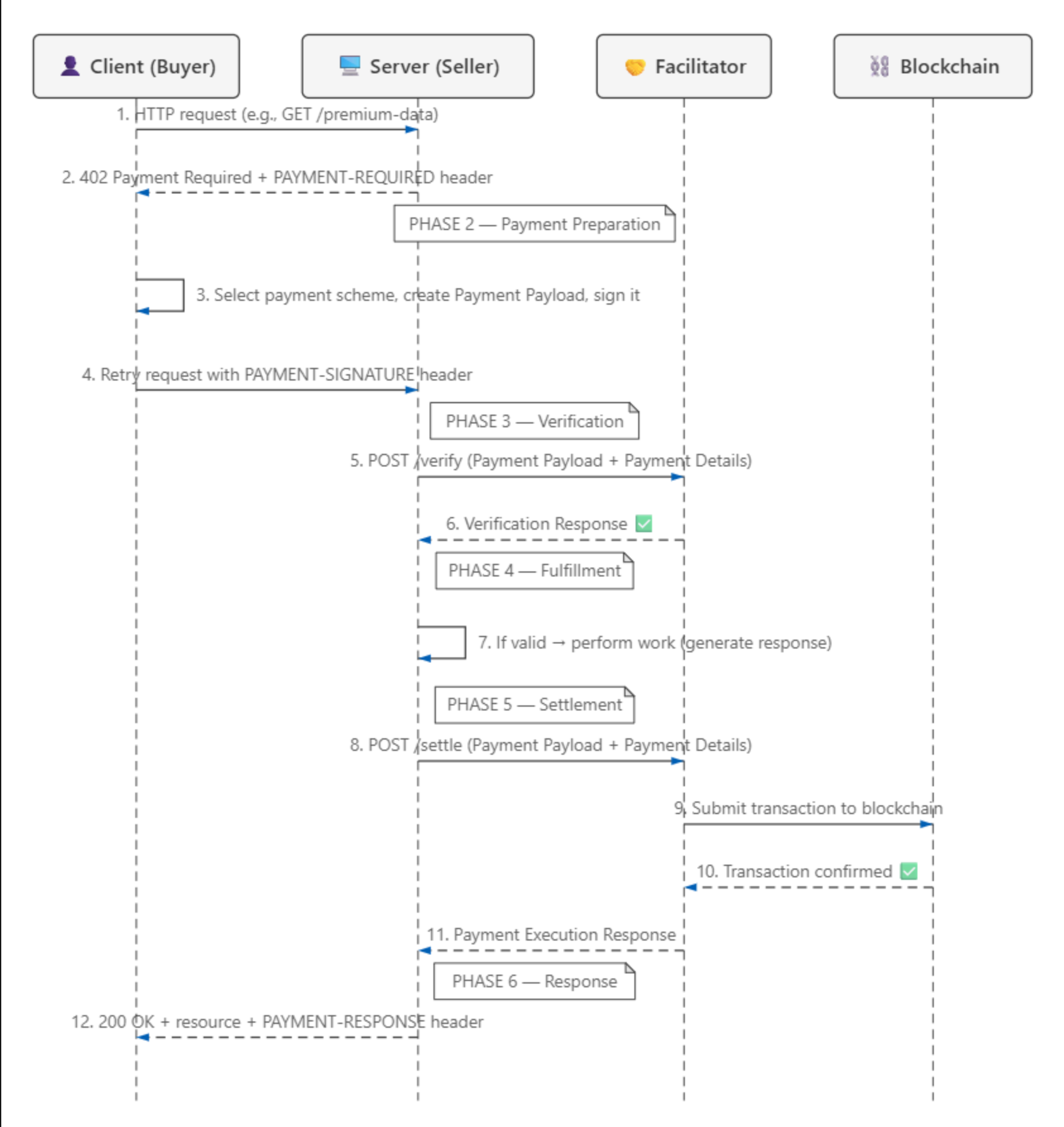
3. Facilitator Responsibilities



Responsibility	What it does	Analogy
Verify	Checks if the client's payment payload is valid (signature, balance, conditions)	Cashier holding a bill up to the light to check if it's real
Settle	Submits the verified payment to the blockchain and waits for confirmation	Cashier depositing the money into the register
Respond	Reports the result back to the server — success or failure	Cashier telling the manager "payment confirmed"

4. End-to-End Interaction Flow

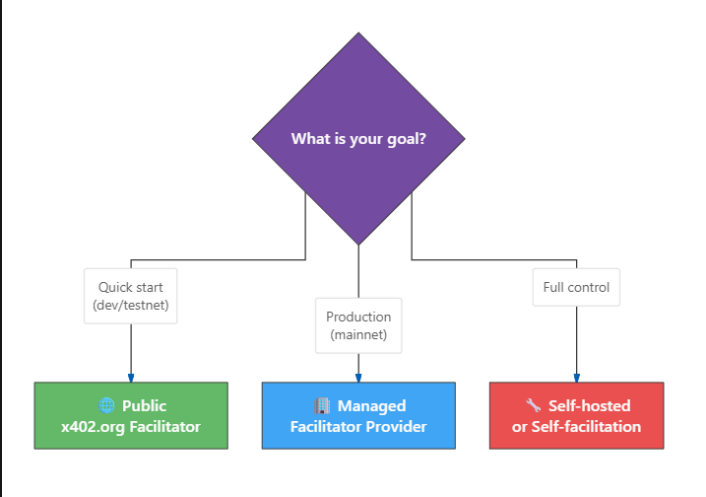
This is the complete lifecycle of a paid HTTP request using a facilitator:



Step-by-Step Breakdown

Step	Actor	Action	Protocol Element
1	Client → Server	Initial HTTP request	Standard HTTP
2	Server → Client	402 + payment requirements (Base64-encoded)	<code>PAYMENT-REQUIRED</code> header
3	Client	Selects a <code>paymentDetails</code> entry, creates payload based on <code>scheme</code>	Payment Payload
4	Client → Server	Retries request with signed payment	<code>PAYMENT-SIGNATURE</code> header
5	Server → Facilitator	Sends payload + details to <code>/verify</code>	Facilitator API
6	Facilitator → Server	Returns verification result	Verification Response
7	Server	Performs work if verification passed	Business logic
8	Server → Facilitator	Sends payload + details to <code>/settle</code>	Facilitator API
9-10	Facilitator ↔ Blockchain	Submits tx, waits for confirmation	Onchain settlement
11	Facilitator → Server	Returns settlement result	Payment Execution Response
12	Server → Client	Returns resource + settlement receipt	<code>PAYMENT-RESPONSE</code> header

5. Choosing a Facilitator Path



Goal	Recommended Path	Notes
Fastest testnet/local quickstart	Public <code>x402.org</code> facilitator	Free, easy, dev/testnet only
Managed production deployment	Production facilitator provider	Supports your target network
Full operational control	Self-host or self-facilitate	See self-facilitation example

!

WARNING: The public `x402.org` facilitator is intended for **development and testnet** only. Do not use it for production mainnet deployments.

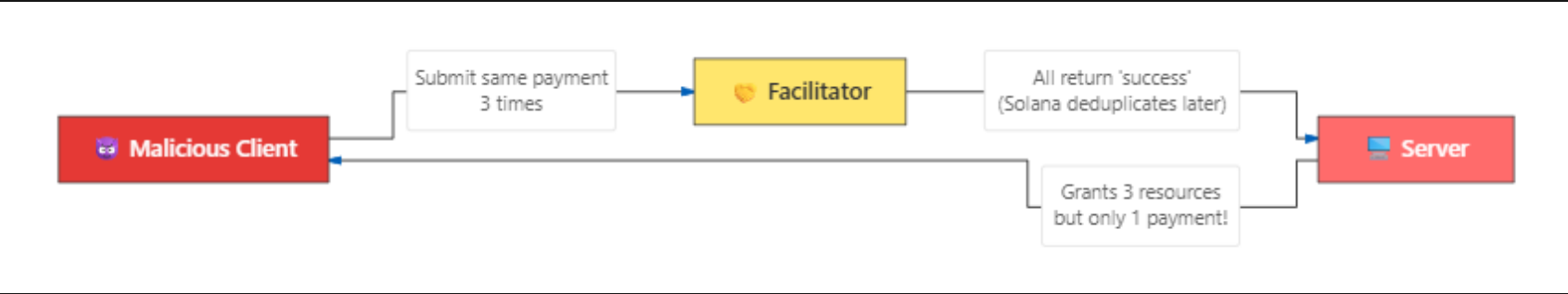
6. Live Facilitators

Multiple facilitators are live in production, supporting various networks:

- **Base** (EVM)
- **Solana** (SVM)
- **Polygon** (EVM)
- **Avalanche** (EVM)
- And more — see [Facilitators directory](#)

7. Solana Duplicate Settlement Protection

On Solana, a **race condition** can occur when the same payment transaction is submitted to `/settle` multiple times before the first is confirmed onchain.



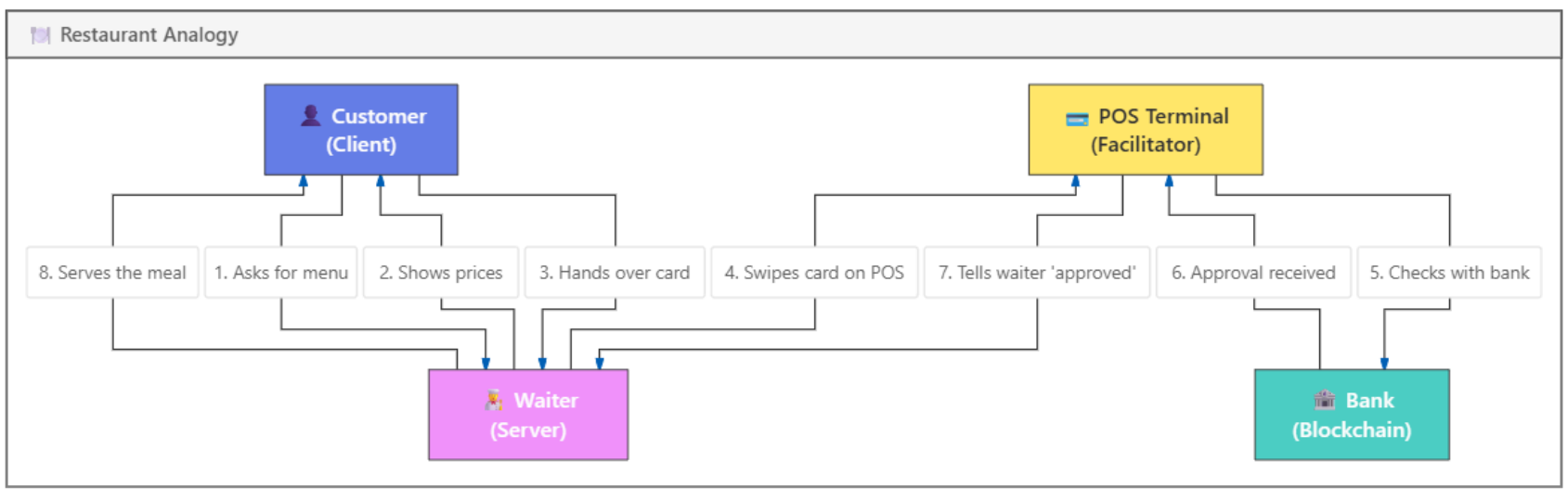
Mitigation: The x402 SVM packages include a built-in `SettlementCache`:

- Short-lived in-memory cache (120 seconds)
- Detects and rejects duplicate settlement attempts
- **Enabled by default** in TypeScript and Python
- In Go, a shared `SettlementCache` instance should be passed during registration

!

CAUTION: If you are a merchant settling payments directly (without a facilitator), you **must** implement equivalent duplicate detection yourself.

8. Restaurant Analogy



Restaurant	x402 Protocol
Customer hands over card	Client sends signed payment payload
Waiter swipes on POS	Server sends verify request to facilitator
POS gets bank approval	Facilitator verifies on blockchain
Meal is served	Resource/data is returned

📖 Glossary

Term	Definition
Client (Buyer)	The party requesting access to a paid resource
Server (Seller)	The party providing the paid resource
Facilitator	Intermediary service that verifies and settles payments on behalf of the server
Verify	Checking whether a payment payload is valid (signature, balance, conditions)
Settle	Submitting a verified payment to the blockchain and waiting for confirmation
402 Payment Required	HTTP status code signaling "this resource requires payment"
Payment Payload	The signed payment data package created by the client
PAYMENT-REQUIRED	HTTP header (server → client) containing Base64-encoded payment requirements
PAYMENT-SIGNATURE	HTTP header (client → server) containing the Base64-encoded signed payment
PAYMENT-RESPONSE	HTTP header (server → client) containing the Base64-encoded settlement result
Settlement Cache	In-memory protection against duplicate settlement (Solana-specific)
Base Sepolia	Ethereum L2 testnet used for development with x402

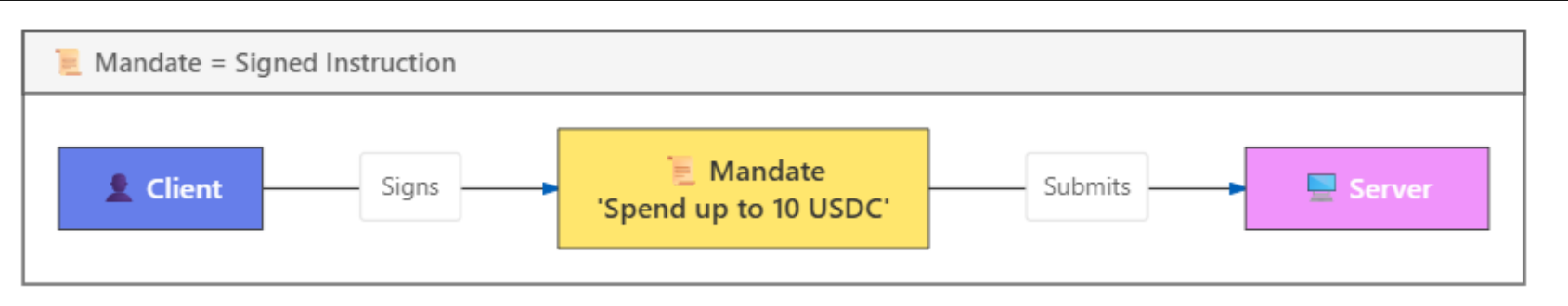
📄 x402'de Mandate, Intent vs Cart ve Authorization ≠ Settlement

1. What is a Mandate?

Mandate = A signed instruction stating: "I authorize spending up to X amount under Y conditions."

Think of a physical check: writing and signing a check is a **mandate**. It is an explicit instruction to the bank saying "pay this person this amount."

In x402, a mandate is the **signed payment instruction** sent by the client (buyer) via their wallet. Funds are not transferred immediately at this point—it is strictly an **authorization/permission**.

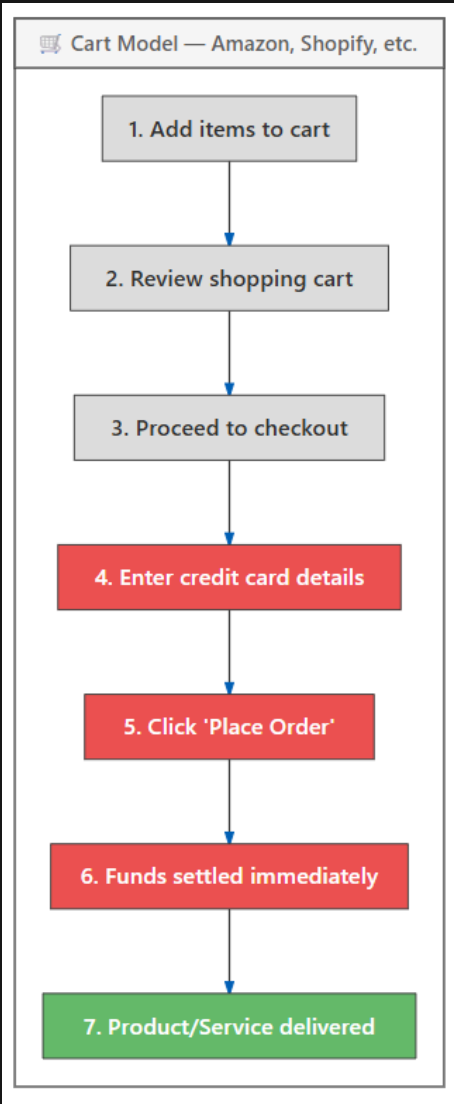


✓ **TIP: Mandate ≠ Payment Settlement.** A mandate is an intent declaration + authorization. The actual on-chain fund transfer occurs later during the settlement phase.

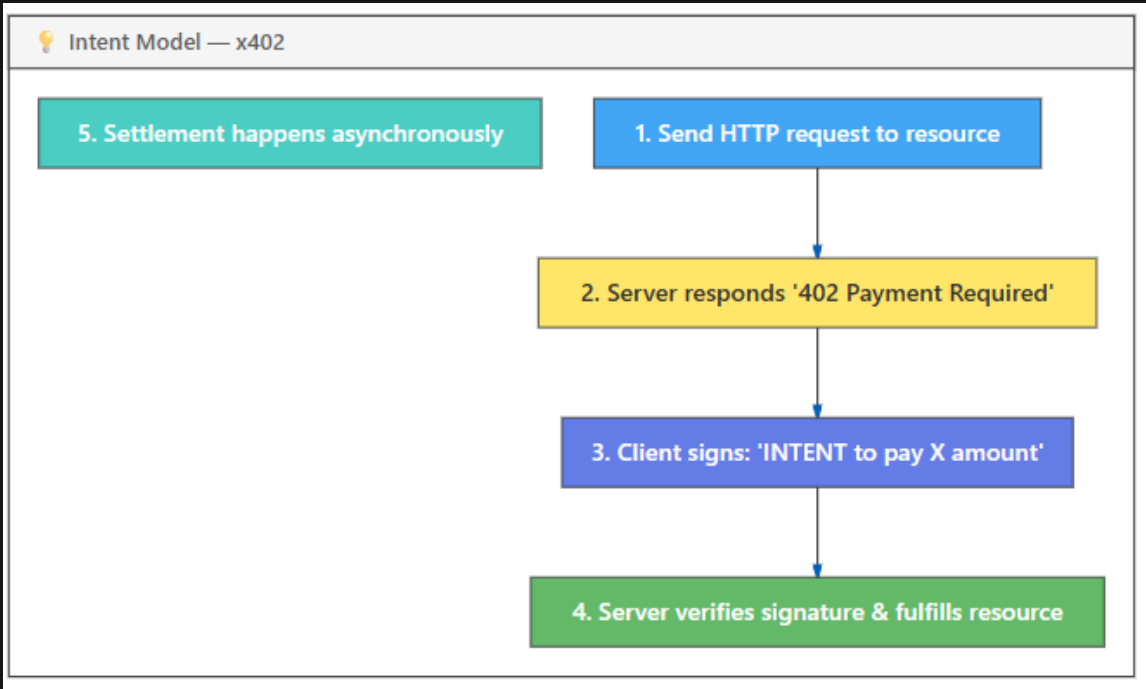
2. ★ Intent vs. Cart — Two Different Paradigms

This is a fundamental architectural distinction separating x402 from traditional e-commerce.

🛒 Cart Model (Traditional E-Commerce)



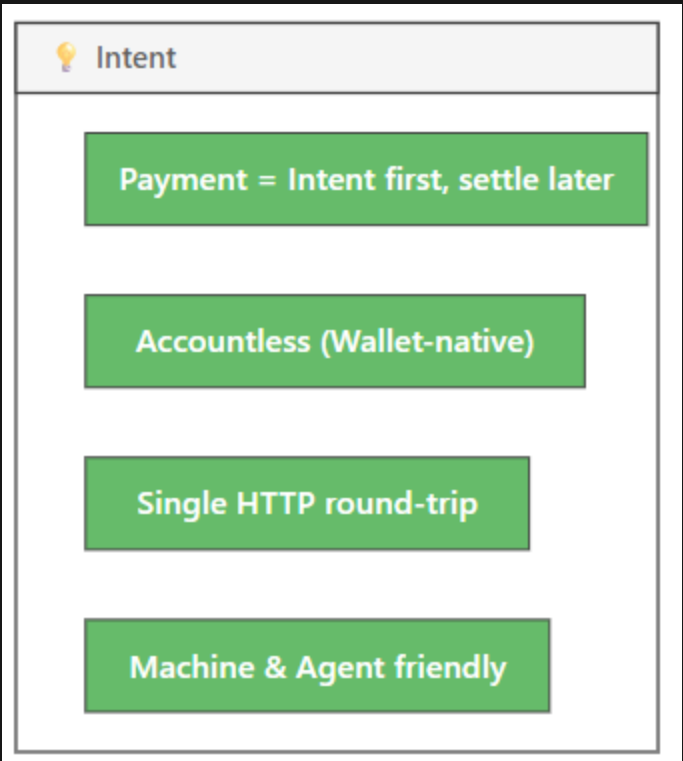
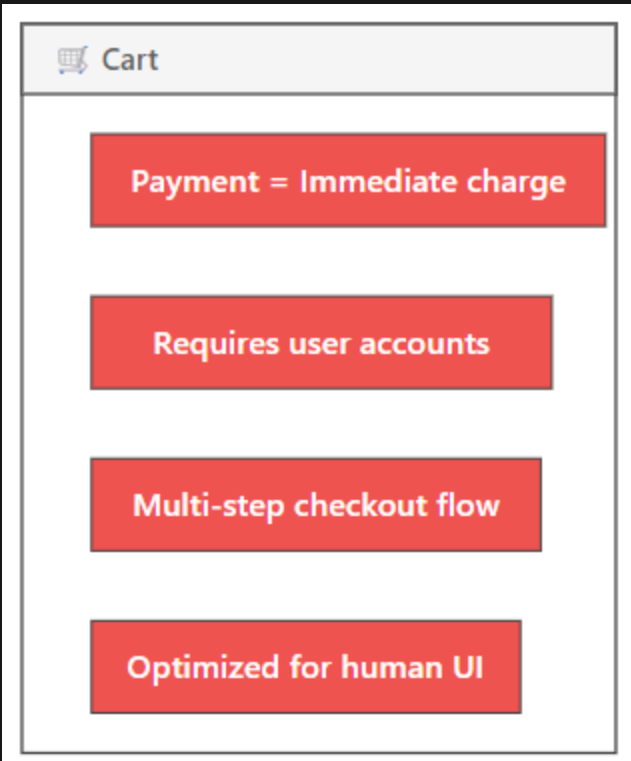
💡 Intent Model (x402)



Drawback: Account creation → Login → Add to cart → Fill shipping info → Enter credit card... **High friction and manual overhead.**

Advantage of mandate: No user accounts, no shopping carts, no session tokens. Simply "Intent to pay → Sign mandate → Access resource".

Comparison Table

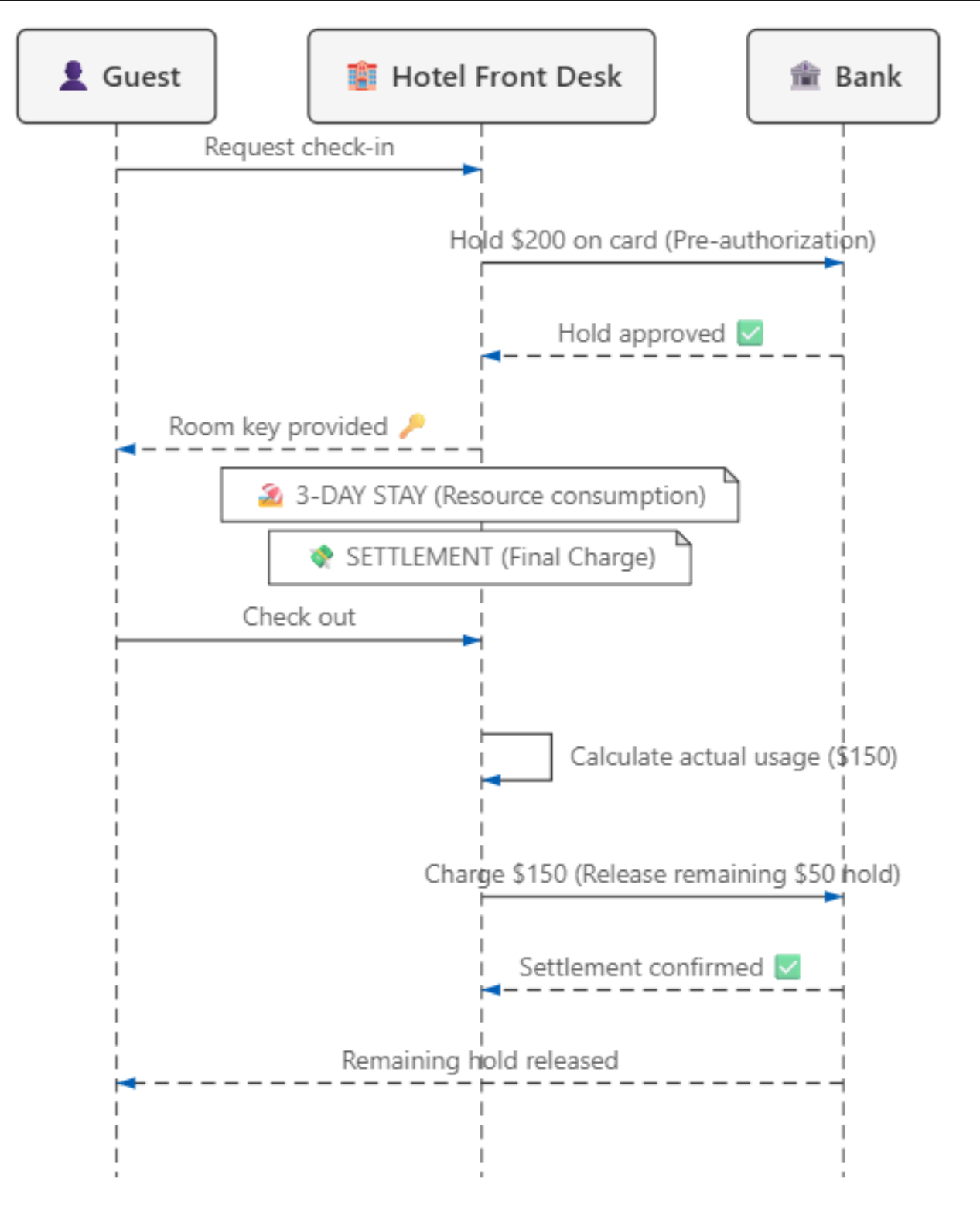


Feature	🛒 Cart (Traditional)	💡 Intent (x402)
Account Required?	✅ Yes (Email, password, profile)	❌ No (Stateless crypto wallet)
Shopping Cart Step?	✅ Yes	❌ No
When are funds moved?	Immediately on button click	Mandate signed → Settled asynchronously
Autonomous AI Agents?	❌ Difficult (CAPTHCAs, complex UIs)	✅ Native (Machine-readable HTTP headers)
Micropayments?	❌ High processing fees	✅ Optimized for sub-cent transactions

3. ⭐⭐ Authorization ≠ Settlement — Why Separate Layers?

This separation is the core architectural concept of x402.

Real-World Analogy: Hotel Check-in



!

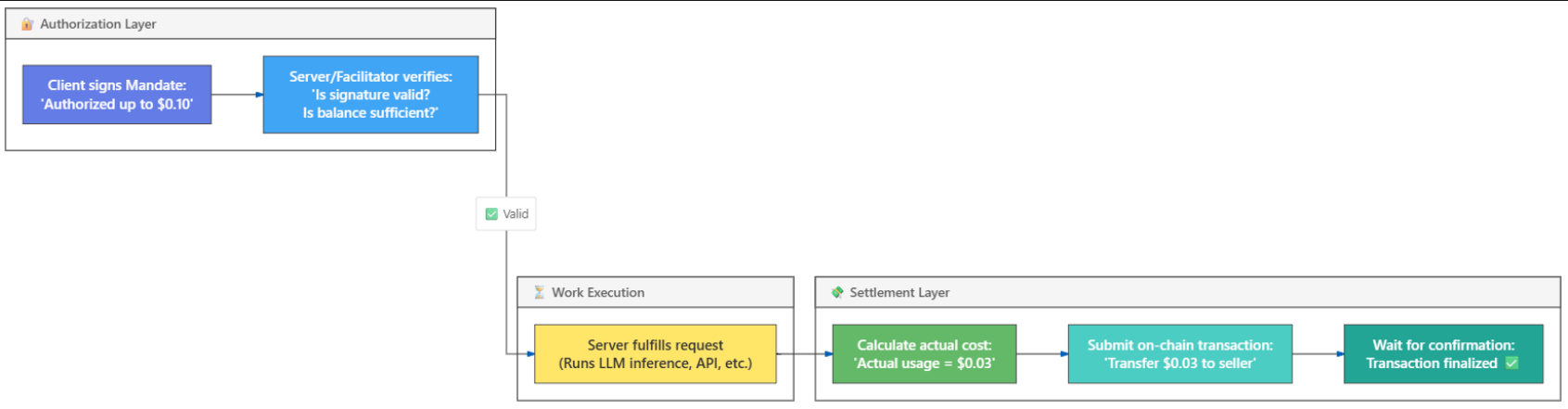
IMPORTANT

Authorization = Pre-authorizing up to a maximum amount → Reserves capacity, does not transfer money

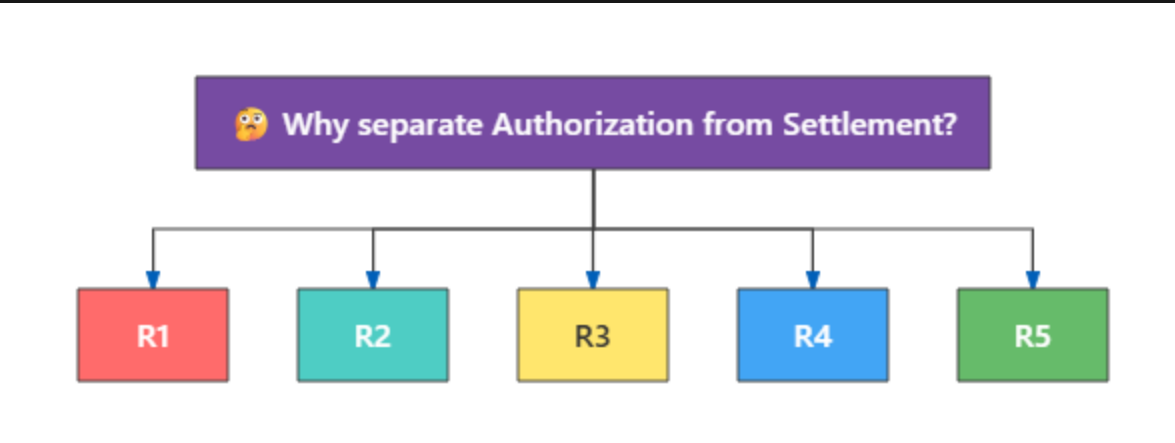
.

Settlement = Submitting the actual final charge on-chain → Executes the real asset transfer.

How This Operates in x402



4. Why Decouple Authorization and Settlement?



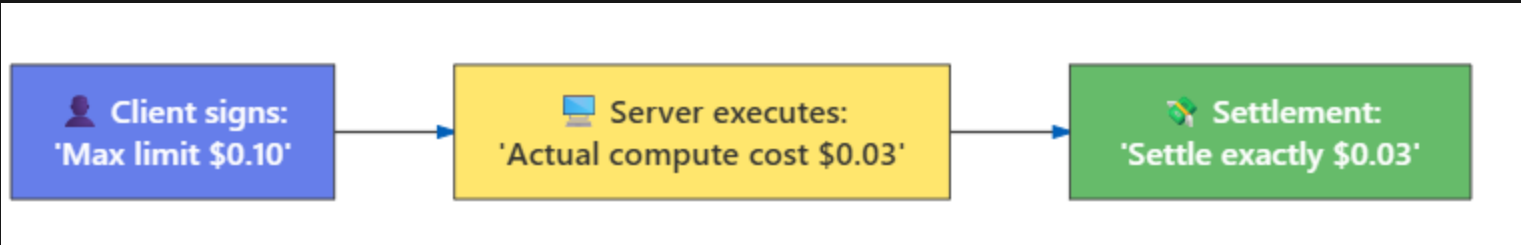
Detailed Breakdown

⚡ 1. Latency Optimization

Phase	Duration
Authorization (Cryptographic Signature Verification)	~Milliseconds
Settlement (On-Chain Block Finality)	~Seconds to Minutes

If an API waited for block confirmation before returning a payload, API responses would be unusable for real-time applications. By decoupling them: **Verify signature** → **Serve resource immediately** → **Settle on-chain asynchronously**.

💰 2. Metered / Pay-As-You-Go Pricing (upto Scheme)

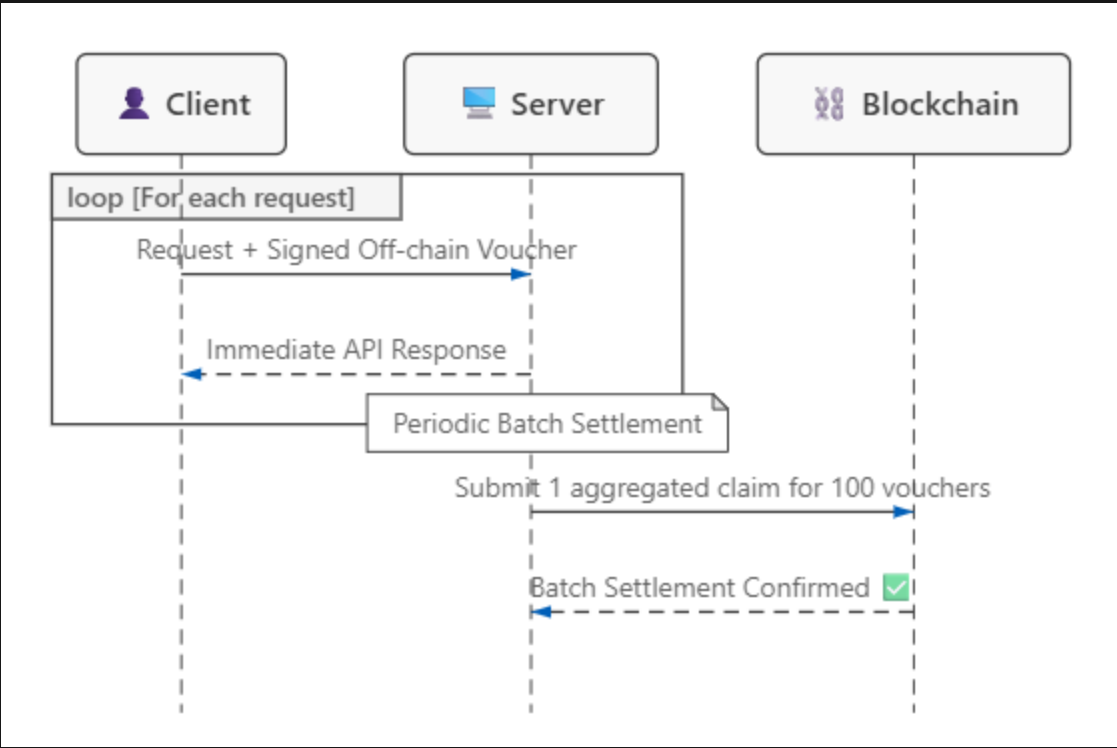


During initial authorization, the final resource consumption (e.g., total LLM output tokens generated) is unknown. The buyer authorizes a maximum budget limit, and the seller settles for the exact consumption.

🛡️ 3. Fraud Prevention & Risk Control

Validating the signed payment payload prior to executing resource-intensive operations ensures that the client possesses the required funds and authority before compute resources are consumed.

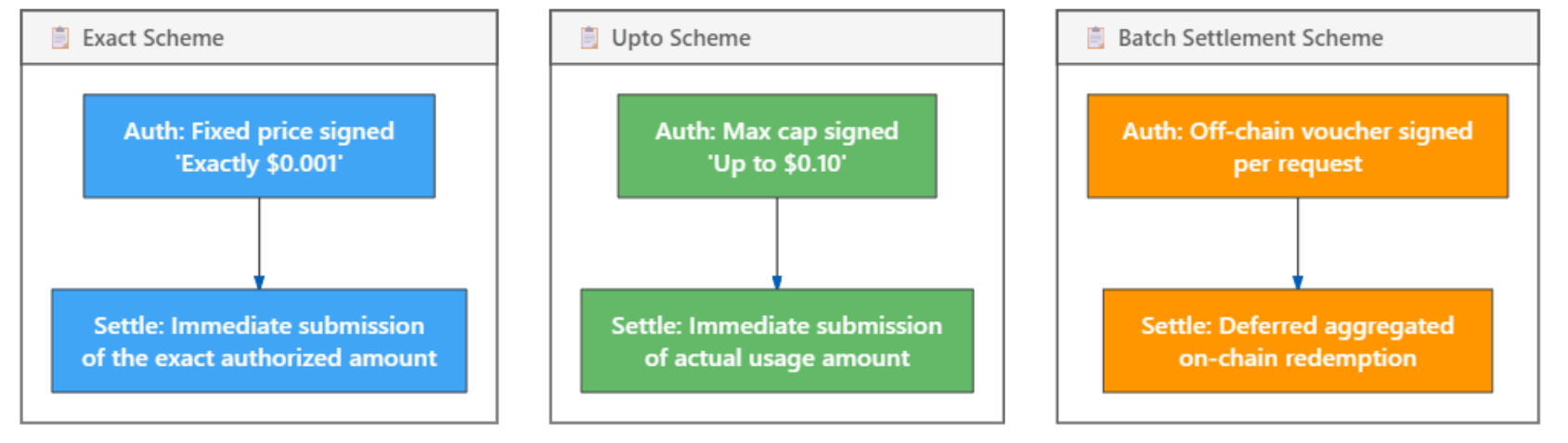
📦 4. Batching Micropayments (batch-settlement Scheme)



Executing an individual on-chain transaction for every sub-cent API call incurs prohibitive gas fees. Decoupling authorization allows off-chain vouchers to be verified instantly and settled on-chain in bulk.

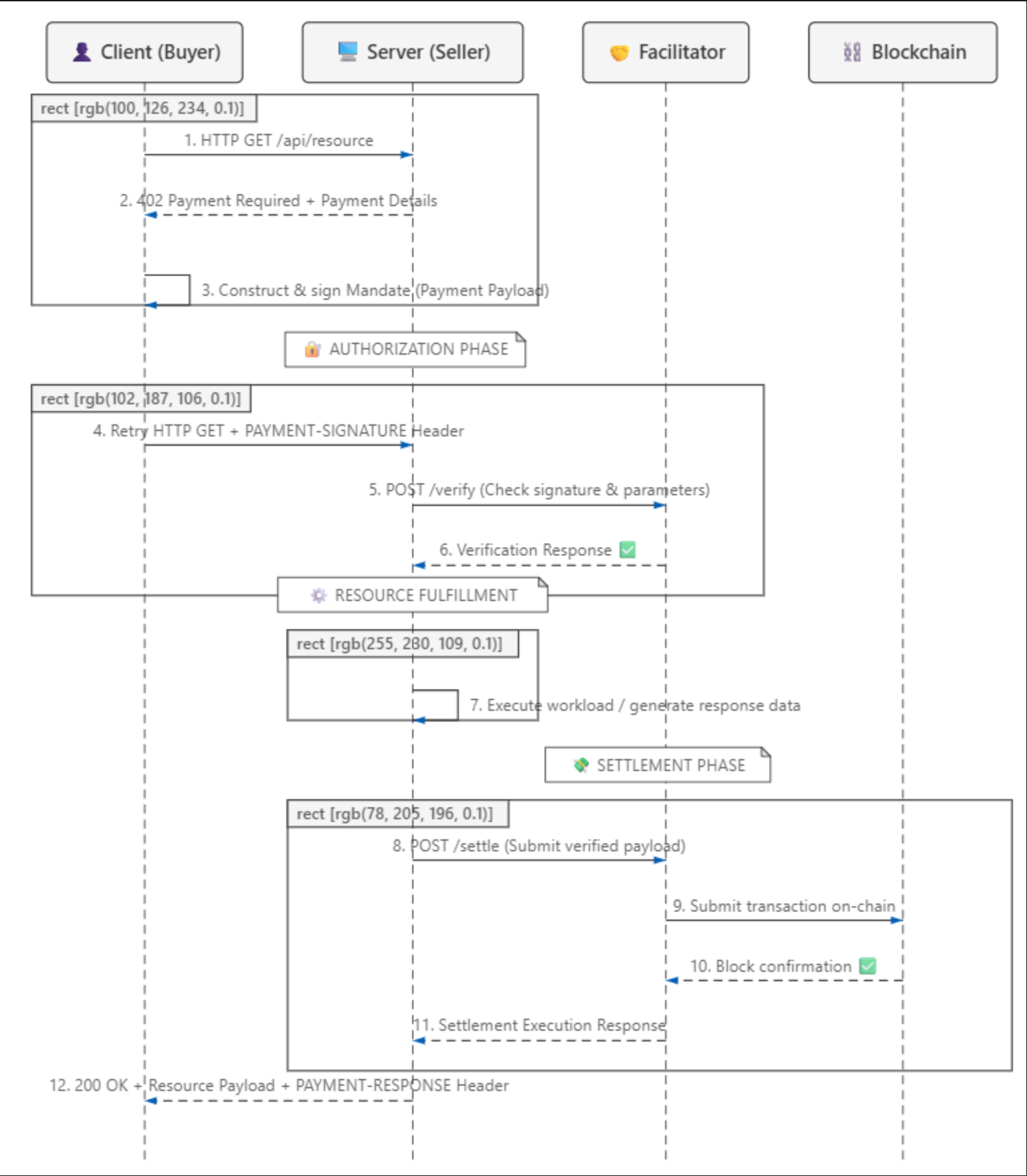
5. Comparison of the 3 Payment Schemes

x402 payment schemes manage the authorization and settlement phases according to different business models:

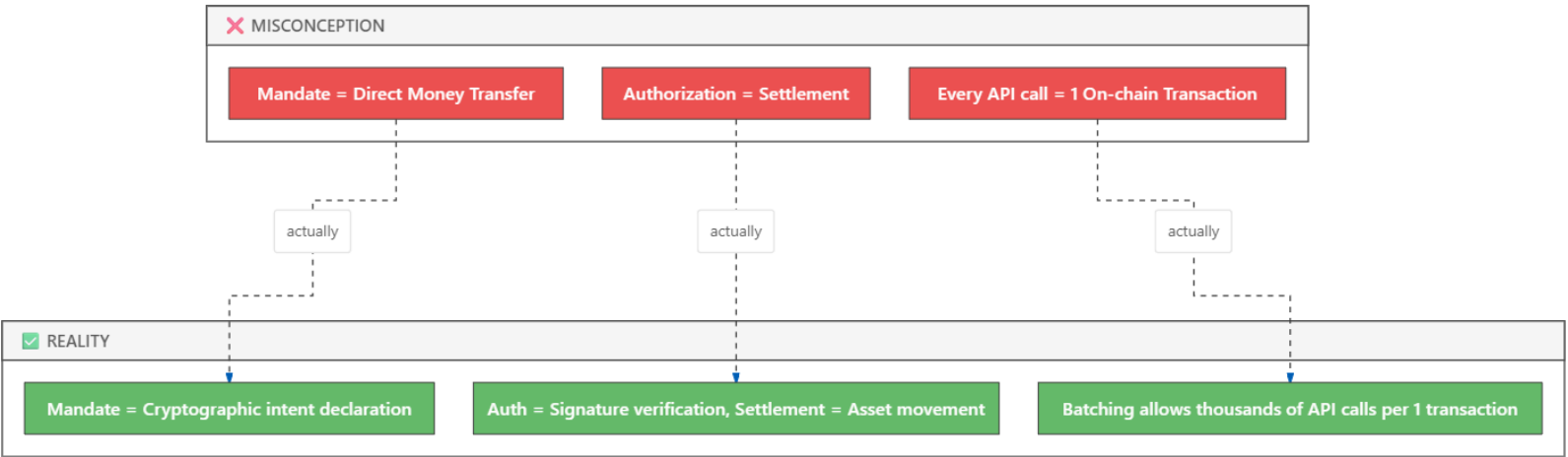


Scheme	Authorization (Auth)	Settlement	Ideal Use Case
Exact	Signs for the exact advertised price	Settles the exact amount immediately	Fixed-price APIs, gated file downloads, paywalled pages
Upto	Signs for a maximum price cap	Settles actual consumption immediately	Usage-based billing (LLM tokens, compute time, bandwidth)
Batch	Signs off-chain vouchers per call	Settles accumulated vouchers periodically	High-frequency API calls, micro-metered streams

6. End-to-End Architecture Flow



7. Common Misconceptions



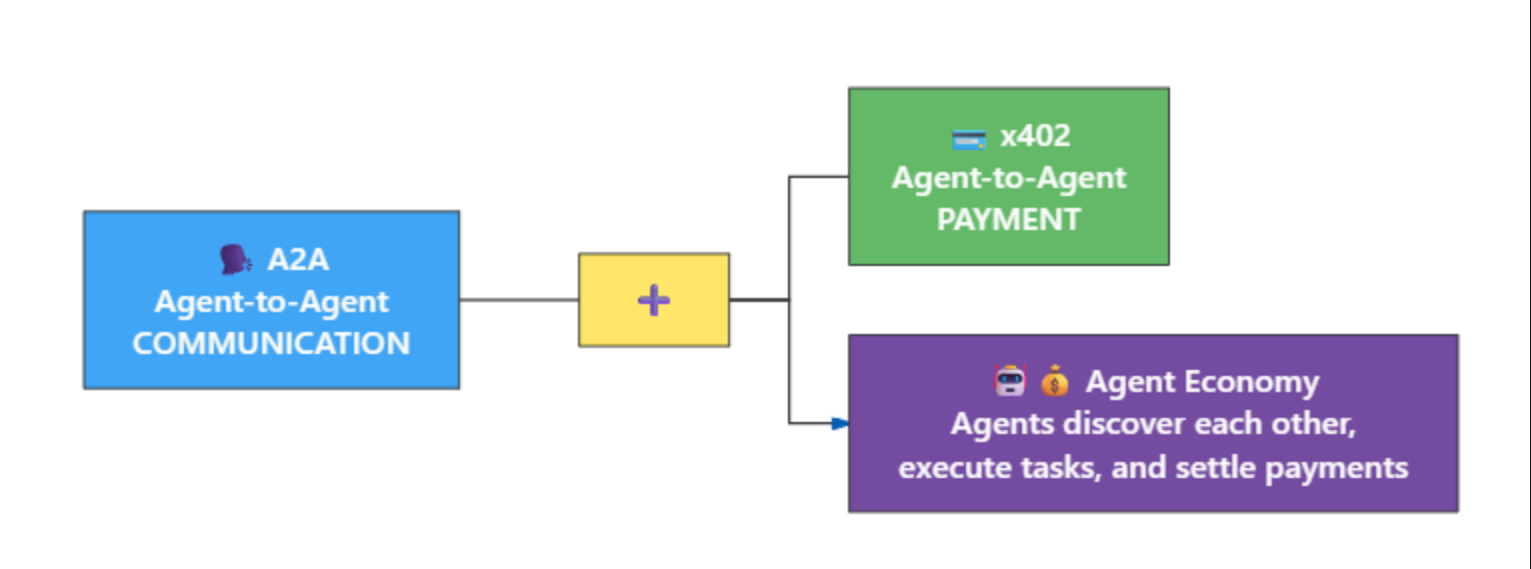
 Glossary

Term	Definition
Mandate	Signed payment authorization parameters specifying spend limits and execution terms
Intent Model	E-commerce pattern where payment authorization is expressed directly within standard HTTP requests
Cart Model	Traditional multi-step checkout flow requiring user accounts and manual checkouts
Authorization	The verification step validating payment signatures, account limits, and terms prior to execution
Settlement	The execution step submitting verified payment instructions to the underlying blockchain ledger
Verify	Facilitator API endpoint validating signed mandates on behalf of resource servers
Settle	Facilitator API endpoint broadcasting approved transactions to the blockchain
Exact Scheme	Payment scheme where authorized amount equals settled amount
Upto Scheme	Payment scheme authorizing a maximum cap and settling actual consumption
Batch Settlement	Channel-based payment scheme aggregating off-chain vouchers into periodic on-chain settlements
Voucher	Cumulative signed off-chain payment authorization used in payment channels
Payment Payload	Base64-encoded JSON structure containing the client's signed mandate

 A2A + x402: How Agents Communicate and Pay Each Other

 Sources: [A2A Protocol](#), [x402 Extensions](#), [x402 HTTP 402](#)

1. The Big Picture — In 3 Seconds

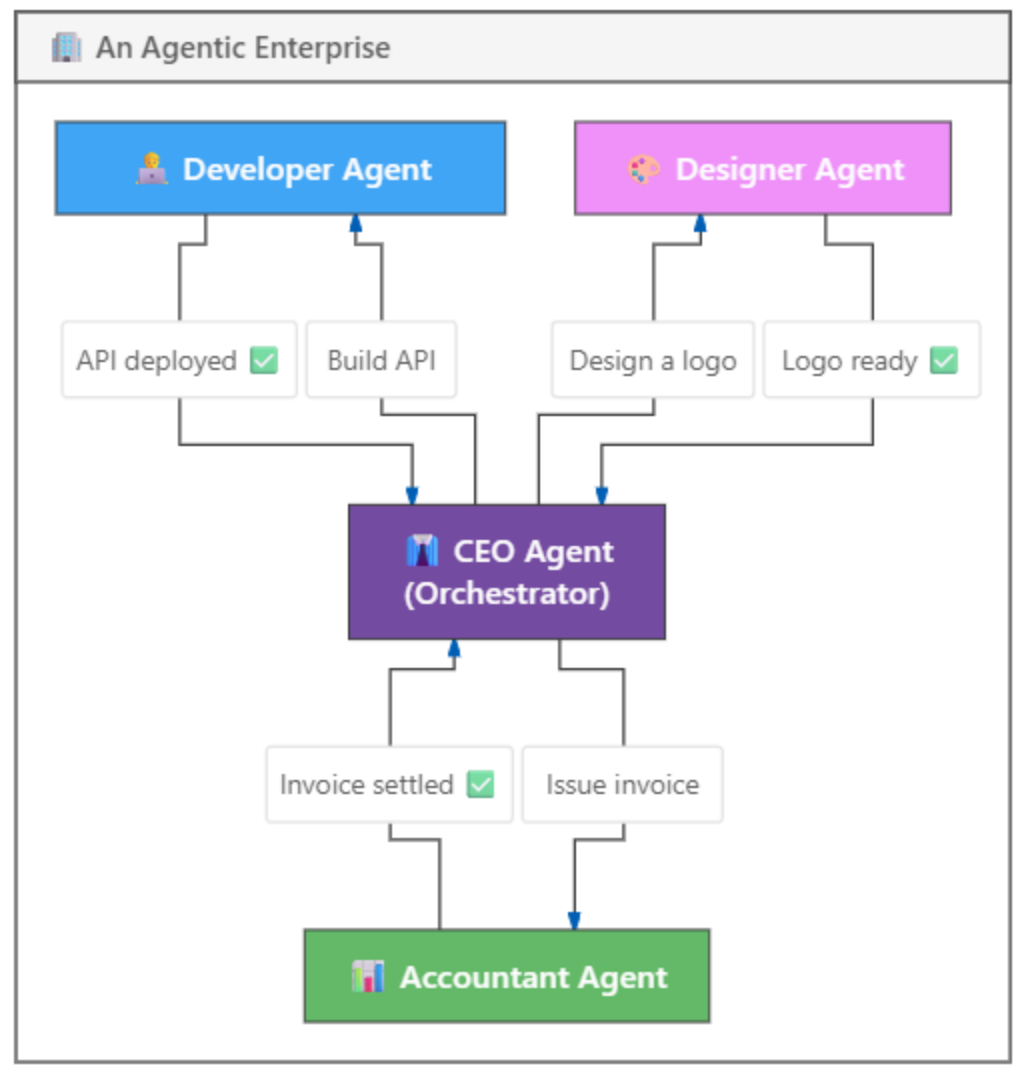


In one sentence: A2A is the **communication language** between agents, while x402 is the **wallet**. Together, they enable agents to autonomously collaborate and monetize services.

2. What is A2A? — Simple Analogy

A2A (**Agent-to-Agent**) is an open standard (originally introduced by Google and donated to the Linux Foundation) that enables AI agents to **discover**, **delegate tasks to**, and **receive outputs from** each other regardless of their underlying framework (LangGraph, CrewAI, BeeAI, or custom code).

 Corporate Office Analogy



Without A2A, these agents speak incompatible proprietary protocols. A2A provides a universal inter-agent communication standard.

TIP

A2A vs. MCP (Model Context Protocol):

- **MCP** = Connects an agent to **external tools & databases** (Local files, SQL, APIs)
- **A2A** = Connects an agent to **other autonomous agents** (Task delegation, multi-agent workflows)

3. ★ Agent Card — The Agent's Business Card

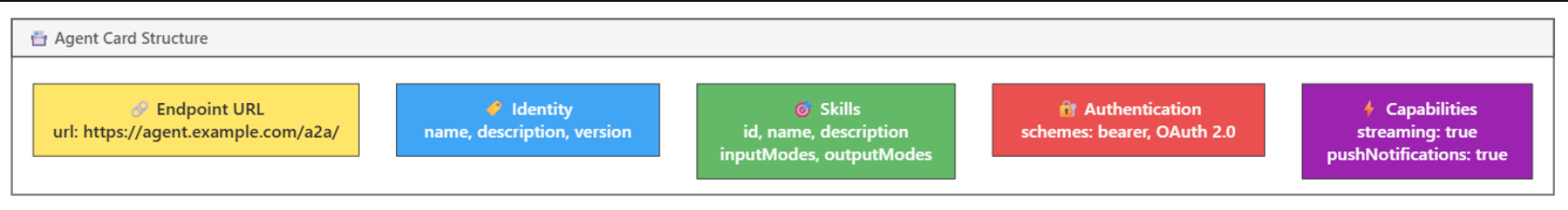
Every A2A agent publishes an **Agent Card**, a JSON metadata document describing its identity, capabilities, endpoints, and security schemes.

Well-Known Endpoint Path

Düz Metin ▾

https://agent.example.com/.well-known/agent.json

Agent Card Architecture



Example agent.json Document

JSON ▾

```
{
  "name": "weather-agent",
  "description": "Autonomous global weather forecasting agent",
  "url": "https://weather-agent.example.com/a2a/",
  "version": "1.0.0",
  "capabilities": {
    "streaming": true,
    "pushNotifications": false
  },
  "defaultInputModes": ["text"],
  "defaultOutputModes": ["text", "data"],
  "skills": [
    {
      "id": "current-weather",
      "name": "Current Weather Query",
      "description": "Returns real-time weather observation for a target location",

```

```
      "inputModes": ["text"],
      "outputModes": ["text", "data"]
    },
    {
      "id": "forecast",
      "name": "5-Day Weather Forecast",
      "description": "Returns 5-day predictive weather metrics",
      "inputModes": ["text"],
      "outputModes": ["text", "data"]
    }
  ],
  "authentication": {
    "schemes": ["bearer"]
  }
}
```

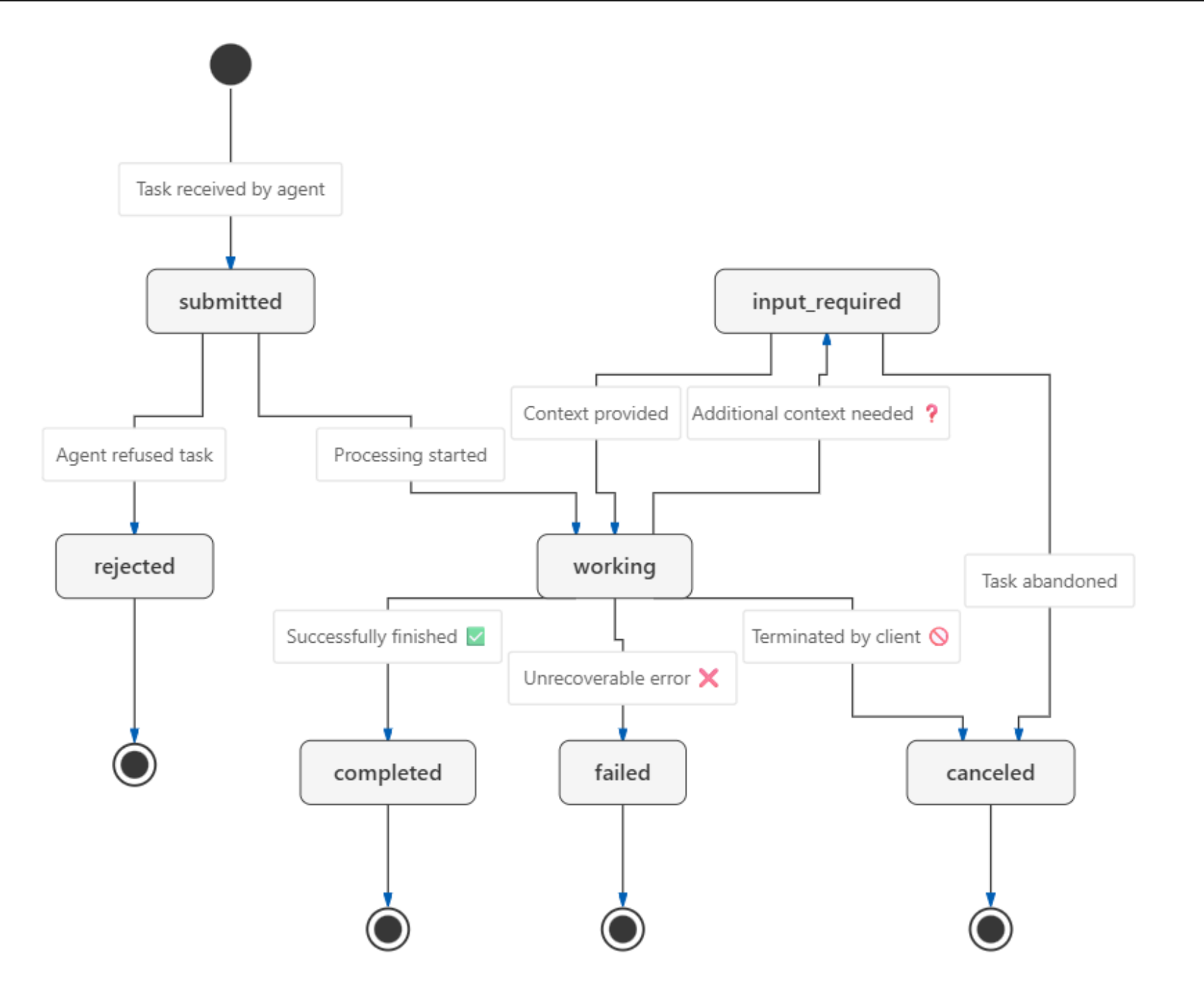
Analogy: LinkedIn Profile

Agent Card Attribute	LinkedIn Equivalent
name	Profile Name / Title
description	About / Summary Section
skills	Listed Professional Skills
url	Contact / Web Endpoint
authentication	Access Requirements / Premium Membership
capabilities	Open-to-work / Messaging settings

4. ★ Task Lifecycle — State Machine

In A2A, work is structured around **Tasks**. Each task moves through a strictly defined state machine.

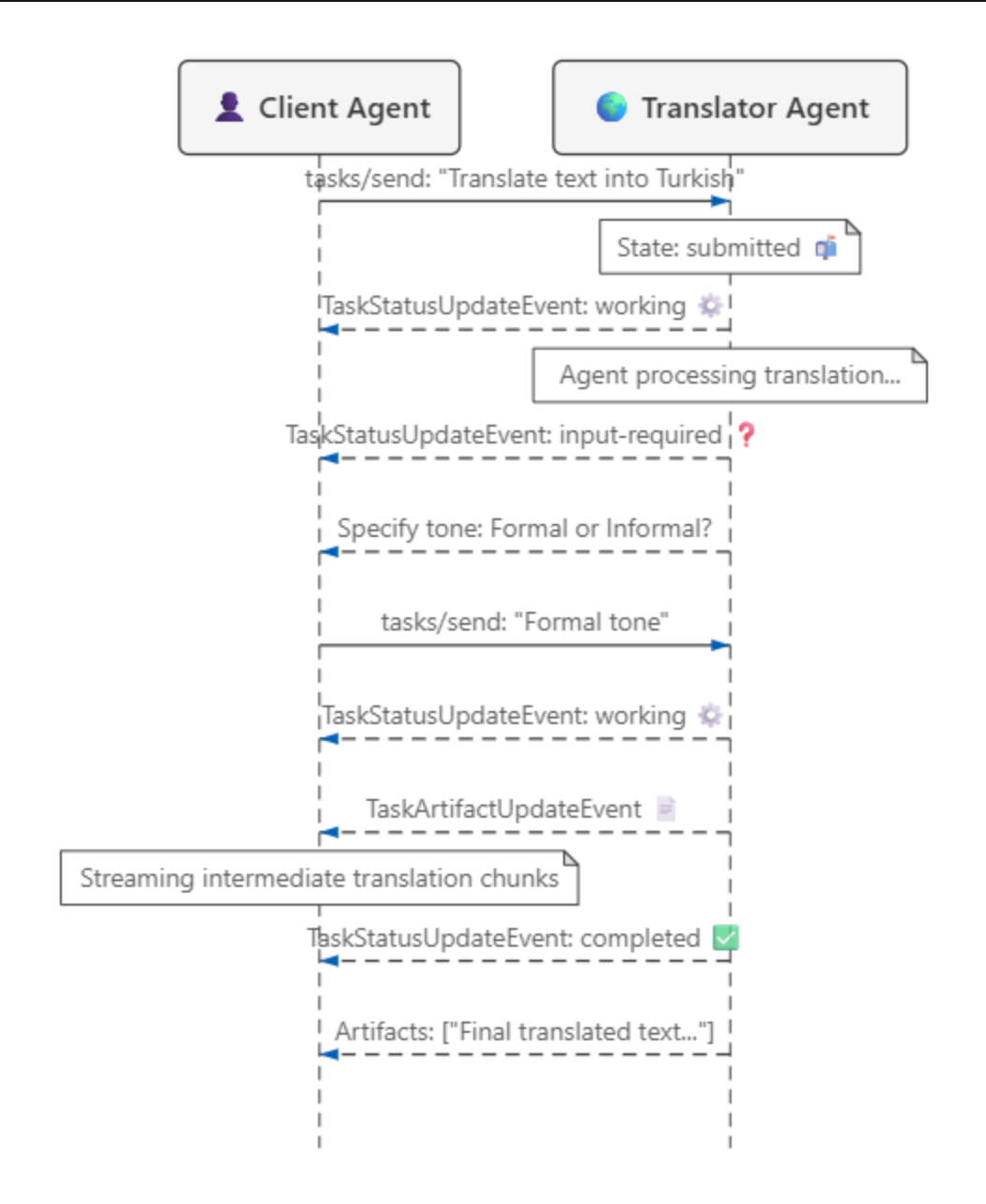
State Diagram



Task State Descriptions

State	Status	Meaning	Real-World Equivalent
submitted		Task received and queued, work not yet started	Order received
working		Agent actively processing work or calling tools	Kitchen preparing food
input - required		Execution paused awaiting clarification/input	"What size/color?"
completed		Execution succeeded, final artifacts available	Package delivered
failed		Task aborted due to runtime error or exception	Out of stock / Error
canceled		Interrupted or aborted by requesting agent	Order canceled
rejected		Refused due to authorization or policy constraints	"Cannot fulfill request"

Message & Artifact Flow: Translation Task Example



Anatomy of an A2A Task Object

Task Structure

 status
submitted / working / completed...

 taskid
Unique task identifier

 contextid
Groups related multi-turn conversations

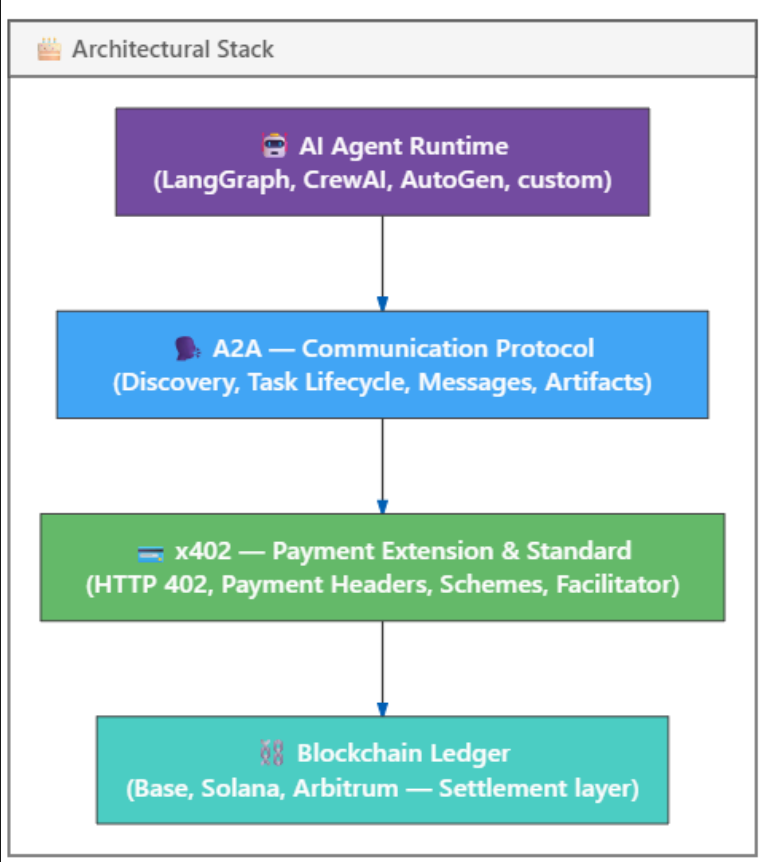
 messages
Full history of exchanged turns

 artifacts
Final deliverables: files, JSON, text

5. ★★ How A2A and x402 Compose

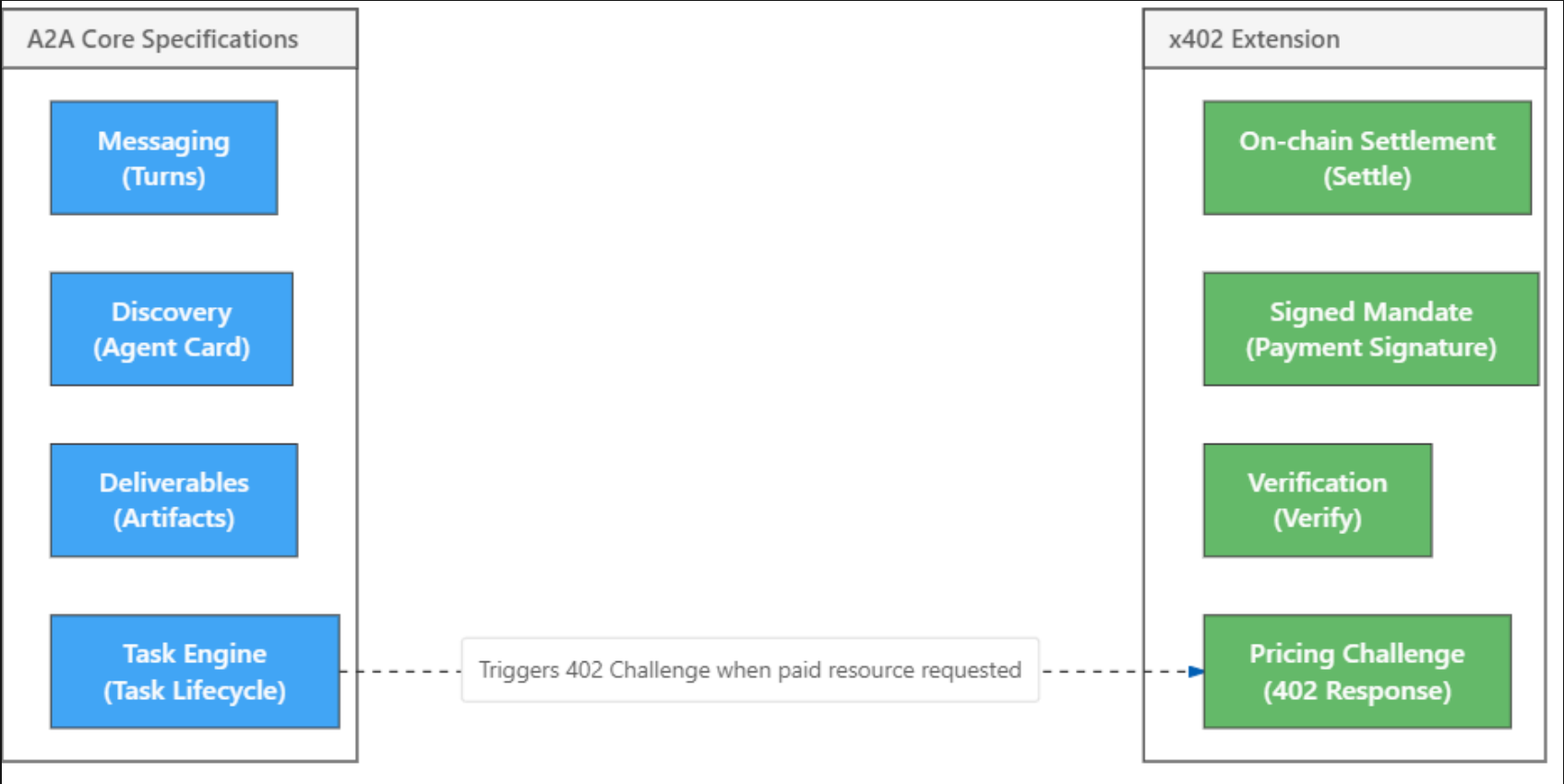
A2A and x402 operate as complementary, composable layers within the agent stack.

Layered Architecture



Layer	Protocol	Primary Responsibility	If Omitted?
Messaging	A2A	Agent discovery, task delegation & execution tracking	Agents cannot interoperate across frameworks
Payment	x402	Machine-readable pricing, authorization & settlement	Services must be offered free of charge
Settlement	Blockchain	Trustless financial settlement (USDC, SOL, ETH)	Requires centralized billing infrastructure

Core Principle: A2A is Payment-Agnostic

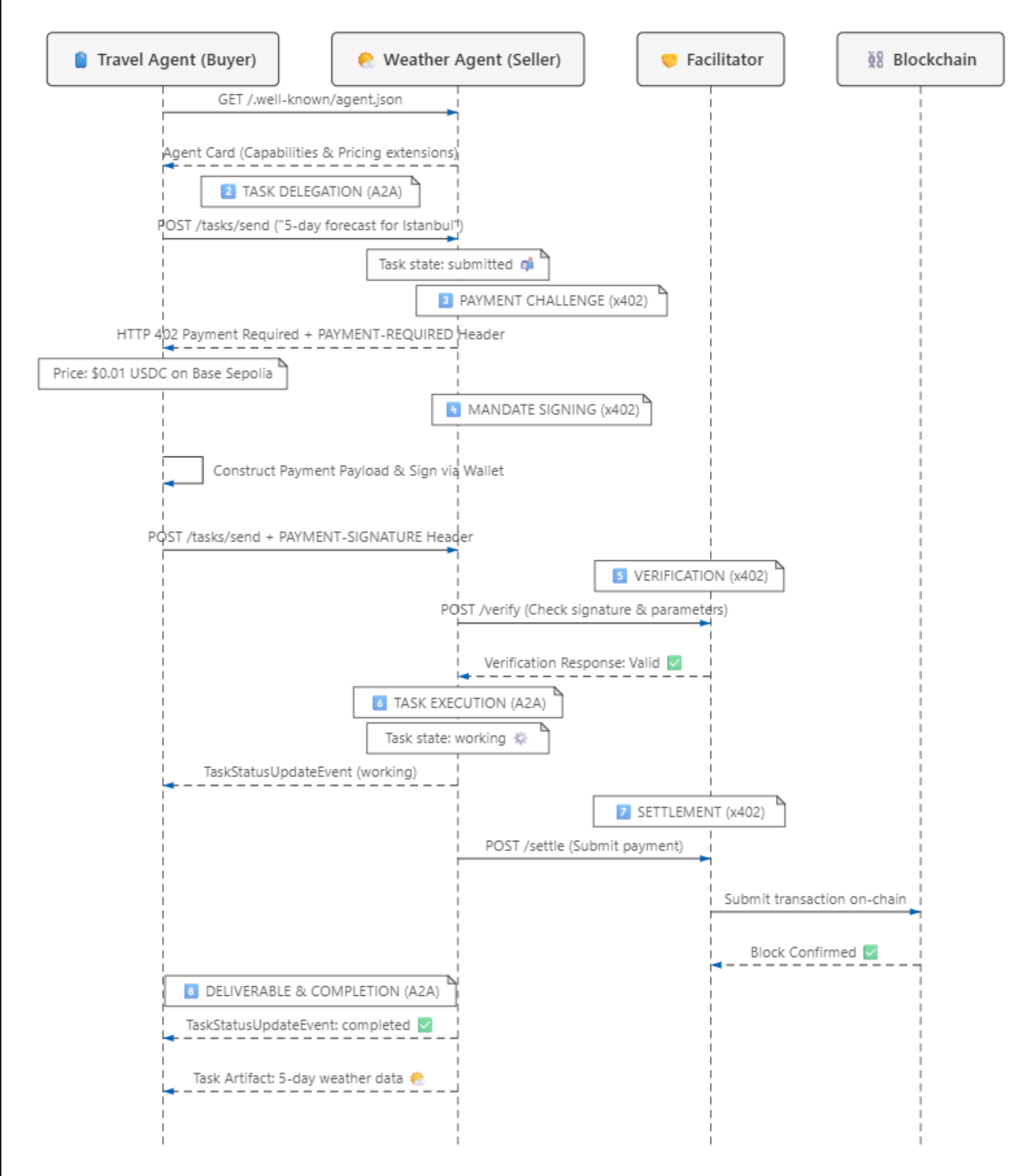


IMPORTANT

A2A core specification **does not embed payment semantics**. It remains payment-agnostic. x402 plugs into A2A endpoints as an **extension**, enforcing payments during task requests via standard HTTP 402 handshakes.

6. ★★ ★ End-to-End Scenario: Agent Hires Another Agent

A Travel Planning Agent delegates a paid task to a Weather Agent:



Step-by-Step Breakdown

Step	Protocol Layer	Action	Description
1	A2A	Discovery	Buyer reads Agent Card to evaluate capabilities and pricing
2	A2A	Task Creation	Buyer submits task; server initializes state as <code>submitted</code>
3	x402	Payment Challenge	Server responds with <code>402 Payment Required</code> and pricing details
4	x402	Mandate Submission	Buyer signs mandate and retries with <code>PAYMENT-SIGNATURE</code> header
5	x402	Verification	Facilitator verifies signature, balance, and network terms
6	A2A	Execution	Server updates status to <code>working</code> and begins execution
7	x402	Settlement	Facilitator broadcasts payment on-chain; funds settle
8	A2A	Artifact Delivery	Server returns generated output as an Artifact; status set to <code>completed</code>

7. Enriching Agent Cards with x402 Metadata

By incorporating the x402 extension into A2A Agent Cards, service prices are declared upfront during discovery:

x402-Enriched Agent Card

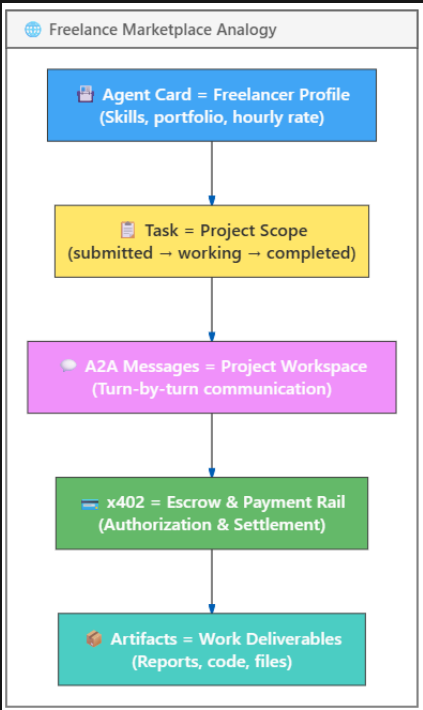
url: https://weather.ai/a2a/

name: weather-agent

skills: current-weather, forecast

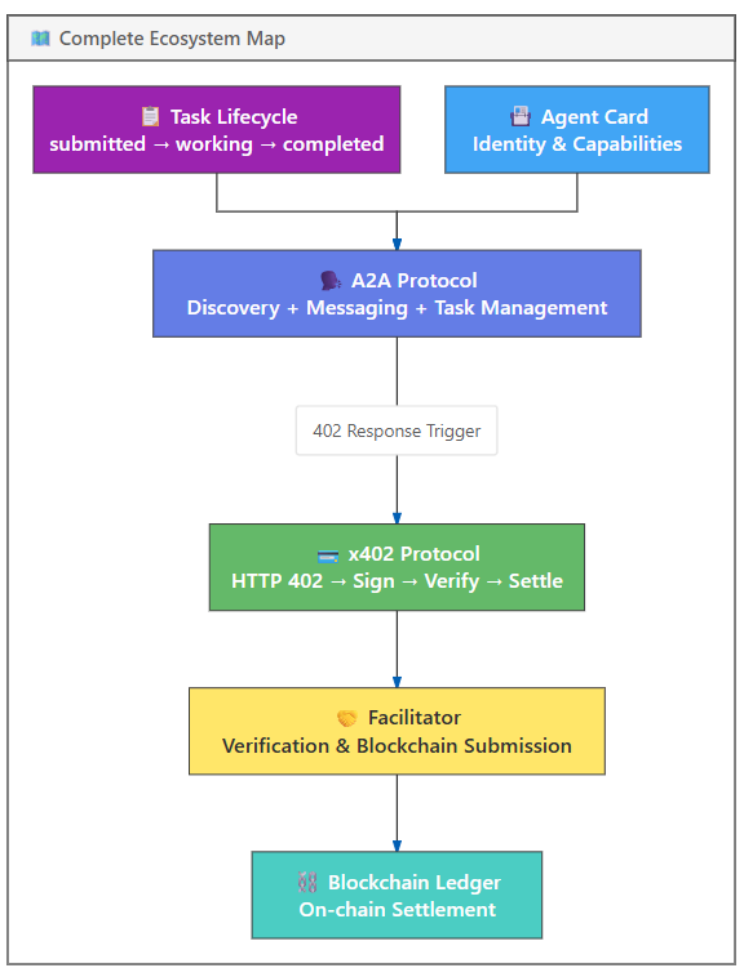
x402 Payment Extension
scheme: exact
price: \$0.01
network: eip155:84532 (Base Sepolia)
asset: USDC

8. Comprehensive Mapping: Upwork vs. A2A + x402



Upwork Feature	A2A + x402 Equivalent
Freelancer Profile	Agent Card (<code>agent.json</code>)
Submit Project Request	Create Task (<code>submitted</code>)
Work in Progress	Task Status (<code>working</code>)
Request Clarification	Task Status (<code>input-required</code>)
Payment & Work Approval	x402 Settlement & Task <code>completed</code>
Cancel Contract	Task Status (<code>canceled</code>)

10. Architectural Summary



Glossary

Term	Definition
A2A	Agent-to-Agent protocol for agent discovery, task management, and communication
Agent Card	JSON metadata file (<code>agent.json</code>) detailing agent identity, skills, and endpoints
Task	The primary unit of stateful work in A2A
TaskState	Status enum representing task progression (<code>submitted</code> , <code>working</code> , <code>completed</code> , <code>failed</code> , etc.)
Artifact	Deliverables returned upon task completion (documents, files, structured data)
contextId	Identifier grouping related tasks and message turns across multi-turn interactions
Skill	A discrete capability declared within an Agent Card
x402 Extension	Composable extension enabling HTTP 402 payment flows within A2A endpoints
MCP	Model Context Protocol — connects agents to local tools and data (distinct from A2A)
Composition	Layering A2A (communication) and x402 (payments) to enable autonomous agent commerce